Operating Systems Week 12 Lecture 12

Junaid Abdul Wahid

Previous

- Virtual Memory
- Demand Paging

Today Agenda

- Thrashing
- Demand Paging

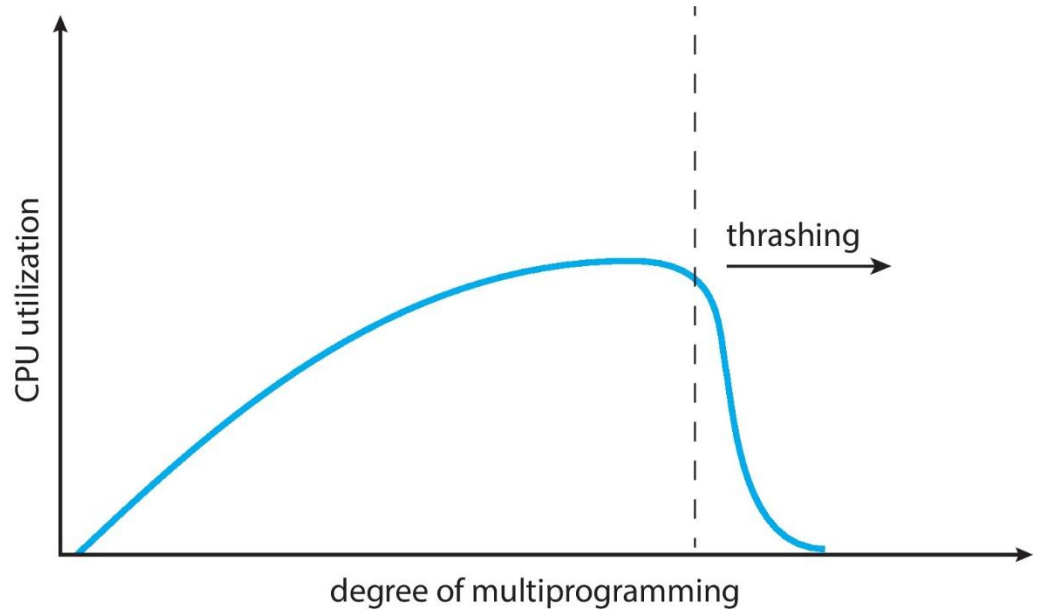
Thrashing

If a process does not have “enough” pages, the page-fault rate is very high

- Page fault to get page
- Replace existing frame
- But quickly need replaced frame back
- This leads to:
  - Low CPU utilization
  - Operating system thinking that it needs to increase the degree of multiprogramming
  - Another process added to the system

Thrashing (Cont.)

Thrashing. A process is busy swapping pages in and out



Thrashing (Cont.)

Thrashing. Thrashing occurs when a process is busy swapping pages in and out of memory instead of executing instructions. This usually happens when there is **insufficient physical memory** and **too many processes** competing for it.

Why Thrashing

- High **degree of multiprogramming**
- **Overcommitment of memory** resources
- Poor **locality of reference**
- Small **working sets** of processes don't fit in memory

Examples

Heavy Browsing

- You open 20+ browser tabs with media-heavy websites.
- Each tab consumes memory.
- System starts swapping between tabs continuously.
- Result: System becomes unresponsive.

Data Analysis with Limited RAM

- Running large datasets in Excel or Python (e.g., Pandas) on a 4 GB RAM system.
- Constant swapping between pages leads to thrashing.

Demand Paging and Thrashing

Why does demand paging work?

Locality model

Process migrates from one locality to another

Localities may overlap

Why does thrashing occur?

Σ size of locality > total memory size

Limit effects by using local or priority page replacement

Working-Set Model (Avoid Thrashing)

The **Working Set Model** is a memory management scheme in operating systems that helps to **avoid thrashing** by tracking the set of pages a process is currently using actively.

- Δ ≡ working-set window ≡ The set of pages(**fixed-size blocks of memory**) a process has referenced in the last Δ time units.
- Δ The window size (measured in time or page references). A tunable parameter.
  Example: 10,000 instructions

WSS_i (working set of Process P_i) = total number of pages referenced in the most recent Δ (time interval)

- if Δ too small will not encompass entire locality
- if Δ too large will encompass several localities

Working Set Example and Goals

Let's say a process accesses pages (**memory blocks**) in this order over time:
1, 2, 3, 4, 1, 2, 5, 1, 2, 3

If we take Δ = 5, then at time t = 10,

W(10, 5) = {1, 2, 3, 5}

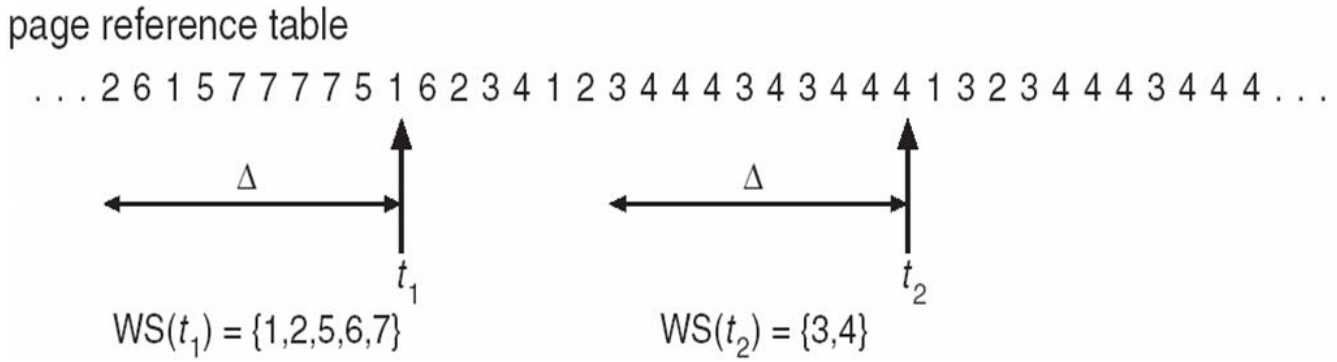
(because these are the pages accessed in the last 5 time units)

Goal of the Working Set Model

- Ensure that each process has enough memory to hold its **entire working set.**
- If not, **suspend** the process or reduce the **degree of multiprogramming.**

Working-Set Model (Cont.)

if $D > m \Rightarrow$ Thrashing
Policy if D > m, then suspend or swap out one of the processes



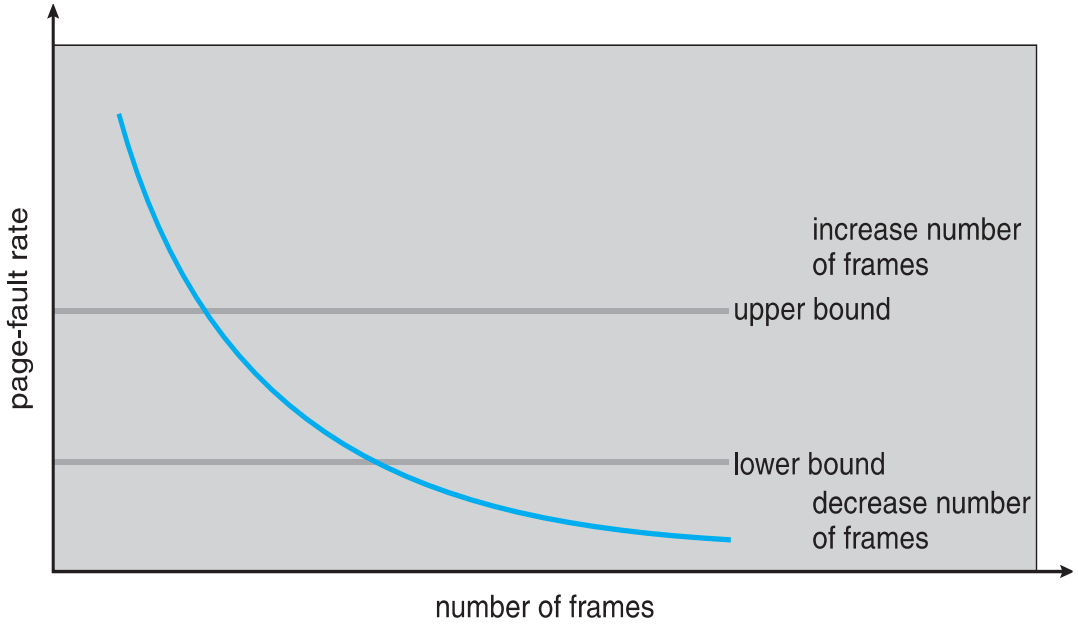
Real Life Example

Imagine you're studying for an exam and keep 5 books open on your table because you switch between them frequently. These 5 books are your *working set*. If your table is too small and you constantly remove and re-place books, your study becomes inefficient—this is similar to **thrashing**.

Page-Fault Frequency

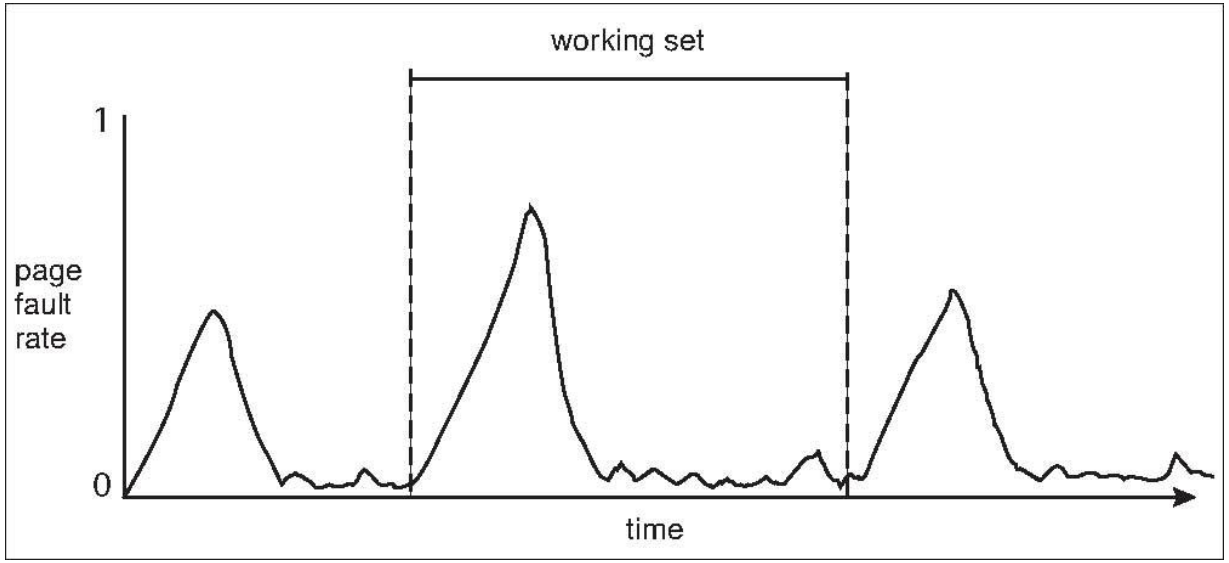
More direct approach than Working set
Establish “acceptable” **page-fault frequency** (**PFF**) rate and use local replacement policy

- If actual rate too low, process loses frame
- If actual rate too high, process gains frame



Working Sets and Page Fault Rates

- Direct relationship between working set of a process and its page-fault rate
- Working set changes over time
- Peaks and valleys over time



Buddy System Memory Allocator

Allocates memory from fixed-size segment consisting of physically-contiguous pages

Memory allocated using **power-of-2 allocator**

- Satisfies requests in units sized as power of 2
- Request rounded up to next highest power of 2
- When smaller allocation needed than is available, current chunk split into two buddies of next-lower power of 2
  - Continue until appropriate sized chunk available

For example, assume 256KB chunk available, kernel requests 21KB

- Split into A_L and A_R of 128KB each
  - One further divided into B_L and B_R of 64KB
    - One further into C_L and C_R of 32KB each – one used to satisfy request

Advantage – quickly **coalesce** unused chunks into larger chunk

Disadvantage - fragmentation

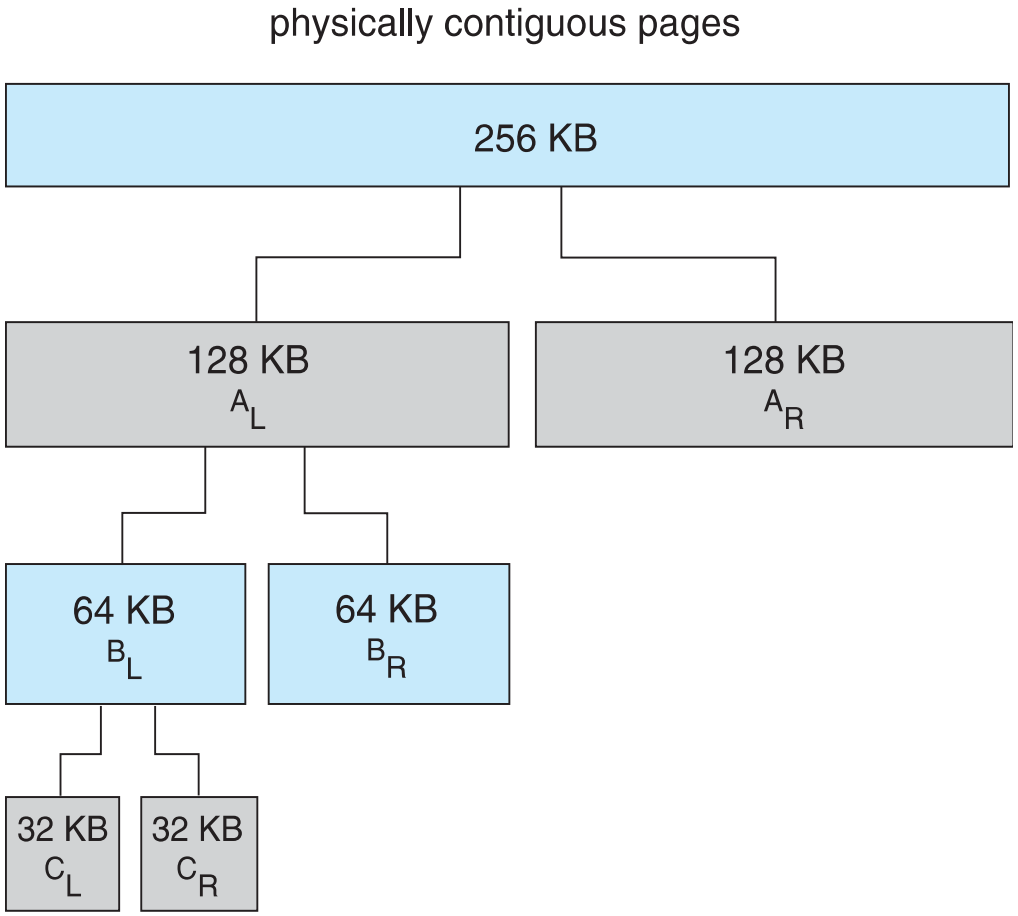
Buddy System Allocator

a **memory allocation algorithm** used by operating systems to **manage free memory efficiently**. It splits memory into partitions to satisfy memory requests, and it allows for easy merging of free blocks.

Suppose Memory is treated as a single large block (typically of size 2^n). It is recursively split into "buddies" (equal-sized blocks) until a suitable block size is found for allocation.

When memory is freed, the system merges the block with its buddy if the buddy is also free.

Buddy System Allocator



Real Life Example

Think of drawers in a cabinet:

- A big drawer can be split into two smaller drawers.
- If you don’t need one, you can join them back again.